1 – Protocolli di rete

La comunicazione a livello *application* si svolge tramite un protocollo ibrido, dove lo scambio dei comandi è svolto tramite l'invio di un messaggio di lunghezza fissa **CMDLEN**; quando si trasmette altri dati (ad esempio una *card*) si opta per un approccio binario per aumentare l'efficienza delle comunicazioni su rete. Tutti i *socket* d'ascolto utilizzati sono bloccanti, in accordo con l'implementazione *thread-based* giustificata nel seguito.

Per le comunicazioni Utente – Lavagna si è deciso di utilizzare il protocollo **TCP** data la necessità di mantenere una connessione persistente tra le due parti e la natura *loss-sensitive* dell'applicazione. Si assume inoltre che la Lavagna abbia risorse computazionali sufficienti per mantenere un numero considerevole di connessioni contemporanee. È comunque presente un *upper bound* per il numero di Utenti potenzialmente connessi alla Lavagna contemporaneamente, dettato dal parametro **MAX_CLIENTS**, per sopperire ad eventuali limitazioni computazionali della Lavagna.

Per l'assegnazione delle attività Utente – Utente si utilizza il protocollo **UDP**, non si vede infatti la necessità di mantenere una connessione ad hoc per ogni coppia di Utenti, che si devono scambiare solo il proprio costo *n* per lo svolgimento dell'attività. Ragionando con la presenza di un numero elevato di Utenti, la complessità computazionale della rete per le sole connessioni Utente – Utente, $O(n^2)$, è difficilmente scalabile. Si vede quindi la necessità di alleggerire il carico di queste comunicazioni. D'altro canto, come già accennato per la connessione con la Lavagna, l'applicativo è *loss-sensitive*. Quindi è necessario implementare un meccanismo basato su ritrasmissione ed *ack* per sopperire alla mancanza del protocollo **UDP** di meccanismi che implementino *reliable data transfer;* nella sezione relativa al *timer* si fa un approfondimento di questo aspetto.

Per garantire un corretto svolgimento dell'asta per le *card*, e per alleggerire il numero di informazioni che ogni Utente deve mantenere bufferizzato, è possibile svolgere solo un'asta alla volta. Un'asta inizierà quando nella colonna **TO_DO** è presente almeno una *card* e sono connessi alla Lavagna almeno due Utenti liberi dallo svolgimento di attività.

2 – Strutture dati e mutua esclusione

Nella Lavagna le strutture dati sono principalmente basate su *liste*, che permettono maggiore scalabilità: le *card* sono allocate in tre *liste* distinte, ognuna rappresentante la relativa colonna del kanban. Allo stesso modo anche i dati relativi agli Utenti attualmente connessi sono allocati in una *lista*.

Data la potenziale presenza di un numero considerevole di Utenti, con comandi che possono essere inoltrati senza vincoli temporali e la presenza di comunicazioni asincrone tra Utenti, una Lavagna concorrente risulta essere la scelta progettuale più diretta: permette, infatti, di dedicare a ciascun Utente un proprio *thread*, parallelizzando la ricezione dei comandi. Lo stesso discorso è applicabile agli Utenti, con un numero considerevole degli stessi risulta ottimale la presenza di *thread* dedicati alla gestione dell'asta in parallelo con la comunicazione con la Lavagna e con la gestione dei *timeout*. Nel seguito è brevemente illustrato il ruolo di ogni *thread*, si rimanda al codice per l'implementazione interna.

La mutua esclusione è implementata tramite quattro semafori di *mutua esclusione* sia per gli Utenti che per la Lavagna, ognuno rappresentante un sottoinseme delle strutture dati. Questo permette di ridurre le attese risultanti dall' implementazione di un singolo semaforo per tutte le strutture dati presenti, aumentanto così la scalabilità del progetto. Non è possibile mantenere più di un *lock* contemporaneamente per mitigare eventuali situazioni di *deadlock* e non si mantiene *lock* durante delle attese su letture bloccanti.

2.1 – Gestione dei timer

Si vede la necessità di introdurre dei meccanismi guidati da *timeout* temporali per migliorare la qualità delle connessioni, per evitare inutili *busy-wait* che rallenterebbero considerevolmente la scalabilità e l'efficenza dell'applicativo. Ho scelto quindi di implementare un meccanismo di *timeout* basato sulla primitiva *alarm* gestita da un apposito *thread* sia per gli Utenti che per la Lavagna. Dato il funzionamento di *alarm*, che permette di tener traccia di un singolo evento temporale alla volta, implemento una *lista* di *timeout* basata sul concetto di *delta-list*. Brevemente, in una *delta-list* ogni elemento mantiene traccia del tempo residuo relativo all'elemento precedente. Solo il *timeout* più prossimo è affiancato da un *alarm*, quando esso genera **SIG_ALRM** il segnale verrà catturato dall'apposito *thread* che rimuoverà l'elemento in testa alla *lista*, settando di conseguenza il nuovo *alarm* con il valore del nuovo elemento di testa, se presente.

L'implementazione del *timer* per la Lavagna è dovuta alla presenza del meccanismo di *ping – pong*, con un tempo d'attesa pari a **TO_PING** e **TO_PONG** rispettivamente. L'implementazione per gli Utenti è simile a quella della Lavagna: si fa ausilio sempre di una *delta-list* ma con inserimento esclusivamente in coda, data la presenza di un solo tempo **TO_ACK**, costante. La coda del *timer* per gli Utenti è utilizzata esclusivamente per garantire la corretta ricezione dei costi delle attività, implementando così il meccanismo di ritrasmissione dei costi delle *card*. Esso è necessario per garantite un corretto svolgimento dell'asta che, lavorando sopra il protocollo **UDP**, non avrebbe altrimenti meccanismi di *recovery* per eventuali *packet loss*, come precedentemente accennato. In caso di *packet loss* relativo ad un *ack*, l'Utente che riceve un *n* duplicato dalla stessa porta, nella stessa contrattazione, reinvia l'*ack* scartando il duplicato.

3 – Lavagna

La lavagna è implementata tramite 3 *thread* principali + 1 *thread* dedicato alla comunicazione con il singolo Utente, di seguito la descrizione del funzionamento di ogni *thread*:

- *alarmHandler* è dedicato alla ricezione ed alla gestione dei segnali di tipo **SIG_ALRM** che implementano il meccanismo di *ping – pong* su cui si basa il monitoraggio delle attività.
- *acceptHandler* è dedicato alla creazione di nuove connessioni, associando ad ognuna un *thread* dedicato.
- *recvHandler* resta in attesa di messaggi da parte dell'Utente, su un *socket* bloccante per evitare *busywait*.
- *terminalHandler* gestisce e resta in attesa di comandi da terminale.

4 – Utenti

Gli Utenti sono implementati tramite 4 *thread* principali, dualmente alla Lavagna:

- *udpHandler* gestisce la comunicazione **UDP** Utente – Utente per lo scambio dei pesi delle *card*.
- *listenHandler* resta in attesa di comandi dalla Lavagna, tramite *socket* bloccante.
- *terminalHandler* gestisce e resta in attesa di comandi da terminale.
- *alarmHandler* è dedicato alla ricezione ed alla gestione dei segnali di tipo **SIG_ALRM** che implementano il meccanismo di ritrasmissione su cui si basa lo *pseudo-rdt* per le comunicazioni **UDP** Utente – Utente.

Nota: per la compilazione è sufficiente eseguire il comando *make*